

# DP5 – Decentralized Namespace

## Claim Your Space in the Meta-Layer

### Purpose of This Draft

This ML-Draft articulates Desirable Property 5 (DP5) as the condition under which people, communities, agents, artifacts, and spaces can be named, addressed, discovered, traded, and governed across the Meta-Layer without dependency on a single platform or registry.

DP5 introduces meta-domains and personal identifiers as sovereign, portable identity and addressability primitives. These identifiers allow participants to claim space in the Meta-Layer, link that space to existing web domains or decentralized identifiers, and use names as anchors for identity, ownership, trust, commerce, and governance.

The core claim is simple:

Meta-domains and personal identifiers give participants sovereign, portable identity – owned by them, not rented from a platform.

The Meta-Layer introduces a decentralized namespace system where identity is not merely a login and addressability is not merely a URL. Names become portable anchors for people, ideas, artifacts, communities, overlays, smart tags, and virtual spaces across the open web.

DP5 guides implementation, governance design, and future ML-RFC development for decentralized naming, meta-domain registration, namespace rights, conflict resolution, and interoperable naming semantics.

---

### 1. Problem Statement: Why Namespaces Matter

The contemporary web depends on naming systems, but most participant-facing names are not truly participant-owned. Handles, usernames, pages, tags, groups, channels, and platform identities are typically rented from centralized services. They can be revoked, shadowed, duplicated, impersonated, renamed, captured, or made non-portable by the platforms that host them.

This creates recurring failures:

- participants build identity and reputation around names they do not control
- communities lose continuity when platforms change rules or shut down access
- artifacts cannot be reliably addressed across tools
- names become vulnerable to spoofing, squatting, seizure, and censorship
- cross-system identity and ownership degrade because identifiers do not preserve meaning across contexts

DP5 reframes naming as civic infrastructure. A decentralized namespace is not only a convenience layer. It is the addressability substrate for identity, agency, data, commerce, interoperability, and governance.

Without DP5, the Meta-Layer cannot reliably answer basic questions:

- What is this person, persona, community, artifact, tag, overlay, or space called?
  - Who controls that name?
  - What does the name resolve to?
  - What rights, policies, and histories are bound to it?
  - How does the name remain interpretable across systems?
- 

## 2. Core Principle of DP5

**Names in the Meta-Layer must be portable, resolvable, governable, and resistant to capture. A namespace that cannot preserve identity, meaning, and control across systems becomes another platform dependency.**

DP5 treats names as more than labels. Names are anchors for participation, reference, ownership, navigation, reputation, and coordination.

A DP5-aligned namespace must therefore support:

- participant-owned identifiers
  - community-owned identifiers
  - artifact and object identifiers
  - namespace portability across tools
  - conflict resolution and reservation logic
  - verifiable ownership and control
  - interpretable resolution across systems
  - resistance to squatting, spoofing, and seizure
- 

## 3. Threats and Failure Modes

### 3.1 Platform-rented identity

Participants build identity around handles or pages that can be revoked, hidden, renamed, or monetized by platform operators.

**Failure mode:** identity continuity depends on platform permission.

### 3.2 Namespace capture

Dominant registries or intermediaries control which names are valid, visible, or resolvable.

**Failure mode:** decentralized naming becomes centralized gatekeeping.

### 3.3 Spoofing and impersonation

Attackers create visually, semantically, or structurally similar names to mislead participants.

**Failure mode:** names become attack surfaces for scams and trust abuse.

### 3.4 Squatting and speculative enclosure

Valuable names are claimed not for use, but to extract rents from future participants or communities.

**Failure mode:** addressability becomes enclosure before public value can form.

### 3.5 Semantic drift

A name carries one meaning in one system and a different meaning elsewhere without signaling.

**Failure mode:** identity, trust, or ownership claims are misinterpreted across contexts.

### 3.6 Registry fragmentation

Multiple naming systems emerge without interoperability or conflict-resolution pathways.

**Failure mode:** participants cannot know which namespace claims are authoritative, compatible, or contested.

### 3.7 Artifact ambiguity

Objects, tags, posts, paths, and digital artifacts cannot be reliably referenced across systems.

**Failure mode:** knowledge, provenance, and ownership degrade because identifiers are not stable.

### 3.8 Non-human namespace ambiguity

AI agents, organizations, bots, and autonomous systems operate without clear namespace rights or management structures.

**Failure mode:** non-human actors become hard to distinguish, govern, or hold accountable.

---

## 4. Primary Namespace Objects

### 4.1 Meta-domains

Meta-domains address virtual spaces within the Metaweb, similar to how traditional domains address web spaces.

A meta-domain may refer to:

- a participant-controlled overlay space
- a community zone
- a smart-tag namespace
- an application surface
- a virtual or conceptual space
- a bridge between a traditional domain and Meta-Layer objects

Example forms include:

- `boeing.com.web4`
- `apple.com.web4`
- `example.com.meta`
- `<label>.example.com.meta`

Meta-domains can link seamlessly to the broader web while functioning within the Metaweb overlay framework.

## **4.2 Personal identifiers**

Personal identifiers address participants and personas, similar to email addresses, handles, or DIDs, but portable across Meta-Layer contexts.

Example:

- `shiftshapr.web4`
- `@jaime`
- `@jaime/artifact99`

Personal identifiers may be connected to decentralized identifiers (DIDs), credentials, proof-of-humanity mechanisms, or zone-specific identity contexts.

## **4.3 Digital artifact identifiers**

Digital artifacts may serve as identifiers, assets, or NFTs that can be bought, sold, transferred, authenticated, or referenced.

Artifacts may include:

- posts
- paths
- smart tags
- annotations
- media objects
- overlays
- lists

- navigation components
- assertions
- credentials

## 4.4 Name chains

Name chains provide structured, semantic identifiers such as:

- @user/object
- @community/tag
- @publisher/claim

Name chains function as URI-like trust anchors, combining identity verification, object authentication, and conflict resolution pathways.

## 4.5 Decentralized URIs and well-known paths

DP5 also recognizes publisher-controlled decentralized URI patterns, such as:

- /.well-known/trust.txt

These allow trusted data to be anchored under existing publisher-controlled domains without requiring centralized registries.

# 5. Namespace System Layer: Resolution, Ownership, and Trust Semantics

This section is upgraded to define DP5 as a **runtime-resolvable, multi-layer namespace system** rather than a static registry.

A DP5-compliant namespace operates across three coupled layers:

## 5.1 Naming Layer (Syntax)

Defines canonical forms, label rules, and human-readable structure.

Examples:

- <label>.example.com.meta
- @user
- @user/object

Requirement:

- deterministic parsing
- canonical normalization

Failure mode: syntactic ambiguity.

---

## 5.2 Resolution Layer (Mapping)

Maps names → entities (people, agents, artifacts, spaces).

Resolution MUST include:

- multi-source resolvers (not single authority)
- explicit resolver provenance
- deterministic fallback rules
- verifiable resolution outputs

Resolution states MUST be visible:

- verified
- provisional
- disputed
- forked
- quarantined

Failure mode: invisible resolution override.

---

## 5.3 Control Layer (Authority)

Defines who can act on a name.

Control may derive from:

- cryptographic keys (wallets)
- DIDs
- registry attestations
- community governance

Control MUST be:

- transferable
- revocable
- auditable

Failure mode: ghost control (no accountable owner).

---

## 5.4 Meaning Layer (Semantics)

Defines what the name *means* in context.

Includes:

- identity type (human, org, agent, artifact)
- trust signals
- governance policies
- reputation bindings

Meaning MUST:

- travel with the name
- degrade visibly across contexts

Failure mode: semantic drift.

---

## 5.5 Temporal Layer (History)

Names are not static. They evolve.

Systems MUST preserve:

- ownership history
- resolution changes
- dispute timelines
- transfers

Failure mode: history erasure.

---

These five layers together define a **full-stack namespace**. Most systems today only implement 1–2 layers. DP5 requires all five.

---

## 5.10 Name Resolution Protocol (Reference Model)

DP5 requires a shared resolution model so names can be interpreted consistently across tools, overlays, registries, and communities.

A reference resolution flow SHOULD include:

1. **Normalize** the name into canonical form.
2. **Validate** syntax, reserved status, and structural constraints.
3. **Query** one or more resolvers or registries.
4. **Return state**: verified, provisional, disputed, forked, quarantined, retired, or unresolved.
5. **Attach provenance**: resolver source, timestamp, signatures, registry snapshot, and confidence level.
6. **Apply local policy**: zone rules, trust profiles, community reservations, and agent constraints.

7. **Render interface signal:** show the participant what the name means, what is uncertain, and what actions are safe.

Resolution must therefore be understood as a governed process, not a hidden lookup.

A minimal response object SHOULD include:

```
{
  "name": "@jaime/artifact99",
  "canonical": "@jaime/artifact99",
  "entity_type": "artifact",
  "controller": "did:example:123",
  "state": "verified",
  "resolver": "registry.example.meta",
  "resolved_at": "2026-04-26T00:00:00Z",
  "provenance": {
    "snapshot": "graph-snapshot-v12.json",
    "checksum": "sha256:...",
    "signatures": ["..."]
  },
  "policy": {
    "transferable": true,
    "dispute_status": "none",
    "zone_constraints": []
  }
}
```

Failure mode: **opaque resolution**, where a name appears trustworthy but participants cannot inspect how that trust was produced.

## 5.11 Namespace State Machine

DP5-aligned systems SHOULD expose name lifecycle states explicitly.

A name may move through the following states:

```
available → claimed → provisional → verified → active
                        ↘ disputed → resolved
                        ↘ quarantined → restored / retired
active → transferred → active
active → expired / abandoned → available or archived
active → forked → parallel-governed state
```

Core states:

- **available:** no active claim exists



- **claimed:** a participant or community has initiated registration
- **provisional:** claim exists but is pending verification or policy review
- **verified:** claim has met required proof conditions
- **active:** name is resolvable and usable
- **disputed:** competing claims or safety concerns exist
- **quarantined:** name is constrained due to suspected abuse, spoofing, or policy conflict
- **transferred:** control has changed with receipt
- **retired:** name is no longer active but history is preserved
- **forked:** multiple governance contexts recognize different meanings or controllers

State transitions **MUST** produce receipts where they affect ownership, resolution, trust, or participant expectations.

Failure mode: **state invisibility**, where participants cannot tell whether a name is safe, contested, provisional, or compromised.

## 5.12 Namespace Attack Taxonomy

DP5 treats names as attack surfaces. Naming systems concentrate trust, discovery, ownership, and memory, making them attractive targets for adversarial actors.

Common attacks include:

### 5.12.1 Spoofing

Attackers register visually or semantically similar names to impersonate trusted entities.

Examples:

- `appIe.meta` using confusing characters
- `paypaI.web4`
- near-match community names

Required response: similarity detection, warnings, and dispute pathways.

### 5.12.2 Squatting

Actors claim names for speculative rent extraction rather than use.

Required response: reservation rules, renewal logic, staking, decay, or community challenge mechanisms.

### 5.12.3 Resolver capture

A resolver becomes a de facto authority by controlling defaults.

Required response: multi-source resolution, resolver provenance, and fallback transparency.

#### **5.12.4 Ownership laundering**

A name is transferred repeatedly to obscure harmful history, evade sanctions, or reset trust.

Required response: transfer receipts and visible lineage.

#### **5.12.5 Semantic hijacking**

A name is used in a new context to imply trust or meaning it did not originally carry.

Required response: semantic profiles and degradation signaling.

#### **5.12.6 Agent impersonation**

Automated or AI actors adopt names that imply human identity, authority, or community status.

Required response: mandatory entity classification and controller binding.

#### **5.12.7 Registry amnesia**

A registry loses or suppresses prior state, disputes, or ownership transitions.

Required response: append-only logs, snapshots, checksums, and independent archival.

#### **5.12.8 Namespace flooding**

Attackers create many names to overwhelm discovery, governance, or trust review.

Required response: rate limits, economic friction, proof thresholds, and anti-spam containment.

### **5.13 Reference Patterns and Compatibility**

DP5 does not replace existing naming systems. It defines how meta-layer naming can interoperate with them.

#### **DNS-style names**

DNS provides global resolution and familiar domain semantics, but is vulnerable to centralized registrar control and does not natively express trust state, provenance, or community governance.

DP5 can use DNS-linked anchors while adding overlay-level trust semantics.

#### **DID-style identifiers**

DIDs support decentralized identity and controller binding, but may be difficult for ordinary participants to read or remember.

DP5 can bind human-readable names to DID-backed controllers.

## **ENS / SNS-style naming**

Blockchain naming systems support ownership and transfer, but often emphasize asset ownership more than contextual governance, dispute visibility, or semantic profiles.

DP5 can learn from these systems while requiring visible state, provenance, and governance.

## **Ordinals / BRC333-style artifacts**

Ordinal inscriptions and BRC333-shaped artifacts can anchor durable metadata and namespace records.

DP5 can use inscription ordering, registry snapshots, and graph facts to support canonicity and provenance.

## **Well-known URI anchors**

Publisher-controlled paths such as `/.well-known/trust.txt` allow existing domain holders to publish trust-relevant metadata without centralized registry dependency.

DP5 can compose these with meta-domain records and overlay resolution.

The goal is not one namespace to rule them all. The goal is interoperable naming with explicit trust semantics.

# **6. Meta-Domain Registry Architecture**

A Meta-Domain Registry may operate as a public ingest and registry service for meta-domains, SNS-shaped JSON, BRC333-related ordinals, and related naming artifacts.

A registry can include:

- block-tip polling
- indexer search through services such as UniSat or pluggable alternatives
- candidate classification
- structural graph facts
- quarantined trust edges
- versioned graph snapshot releases with checksums

Such a registry SHOULD distinguish between:

- structural facts that can be merged freely
- trust assertions that require quarantine or policy review
- local drafts that remain outside public canonical state

## **6.1 Candidate classes**

Candidate logs may include:

- `.meta` tokens

- BRC333-shaped bodies
- SNS-alias JSON candidates
- tag registry records
- anchor records
- name-chain references

## 6.2 Registry outputs

A registry SHOULD provide:

- canonical records
- candidate logs
- graph snapshots
- checksums
- dispute or quarantine markers
- provenance for resolver decisions

## 6.3 Canonicity and ordering

Where ordinal inscriptions are used, canonicity SHOULD be determined by documented ordering rules, such as lowest qualifying inscription number where adopted, rather than arbitrary string ordering.

## 6.4 Pluggable indexers

Registries SHOULD support pluggable indexers and resolution sources so that namespace infrastructure does not depend on a single data provider.

Failure mode: **indexer dependency**, where resolver integrity depends on one commercial or centralized API.

# 7. Registration Rules and Validation

DP5 supports concrete registration rules for product-level namespaces.

## 7.1 Canonical form example

A v1 product canonical form may be:

`<label>.example.com.meta`

Where:

- `<label>` is the registrant-chosen segment
- `example.com.meta` is a configurable suffix or canonical parent

## 7.2 Label rules

A valid label SHOULD satisfy:

- lowercase ASCII letters, digits, and hyphen only (a-z, 0-9, -)
- no underscores or Unicode in v1 unless explicitly expanded later
- must not start or end with -
- length between 1 and 63 characters
- must not be an integer-only label
- must not be reserved

## 7.3 Structural validation regex

Label-only validation:

```
^(?!-)([a-z0-9](:[a-z0-9-]{0,61}[a-z0-9])?)$
```

Full-domain validation example:

```
^(?!-)([a-z0-9](:[a-z0-9-]{0,61}[a-z0-9])?)\.example\.com\.meta$
```

Regex alone is insufficient. Reserved-name and integer checks must be separate.

## 7.4 Recommended validation order

1. Parse `label` as the substring before the configured suffix.
2. Reject if label fails length or charset validation.
3. Reject if label matches `/^\d+$/`.
4. Reject if `label.toLowerCase()` is in the reserved set.
5. Reject if the normalized domain is already in use.
6. Accept and register.

## 7.5 In-use rule

“In use” means that a record already exists with the same normalized domain string under the applicable active status rule.

Storage and comparison SHOULD normalize to lowercase.

---

## 8. Interoperability and Tradeability

Tradeable meta-assets allow participants to exchange digital spaces, objects, and names, fostering virtual commerce and community formation.

However, tradeability must remain bounded by trust, provenance, and governance.

DP5 requires:

- transfer receipts
- provenance preservation
- dispute visibility
- constraints on reserved or protected names
- compatibility with commerce and incentive systems (DP6, DP9)
- alignment with identity and accountability requirements (DP1)

A name may be tradable, but the trust attached to the name cannot be treated as a blank commodity. Reputation, history, and community meaning must remain visible across transfer.

---

## 9. Governance, Accountability, and Agency Surfaces

This section is upgraded to define **interface-level namespace governance**, aligning with the Meta-Layer's overlay architecture.

Namespaces are not governed only in registries. They are governed at the **point of interaction** via overlays, filters, and community rules. ←filecite→turn17file2↑

### 9.1 Participant-facing surfaces

Participants MUST be able to:

- claim names under transparent rules
- see resolution paths (who resolved this name?)
- inspect ownership and controller state
- view trust signals and classification (human, agent, org)
- transfer names with receipts
- initiate disputes
- view dispute status in real time

Failure mode: invisible governance.

---

### 9.2 Overlay-mediated governance

Meta-layer overlays SHOULD expose:

- name provenance tooltips
- impersonation warnings
- namespace conflicts
- transfer history
- community annotations

This turns naming into a **live civic surface**, not a backend database.

Failure mode: governance hidden behind APIs.

---

### 9.3 Community governance powers

Communities **MUST** be able to:

- reserve names
- define namespace policies
- classify entities
- enforce local naming norms
- quarantine suspicious identities
- fork namespace rules when needed

Failure mode: centralized naming authority.

---

### 9.4 Dispute visibility

All conflicts **MUST** be visible as states, not silent overrides:

- competing claims
- impersonation reports
- trademark conflicts
- governance disagreements

Failure mode: silent winner-take-all resolution.

---

### 9.5 Interface as enforcement boundary

DP5 aligns with the principle that governance must live at the interface layer, where users experience identity and trust.

Browser overlays act as **civic membranes** where naming rules become visible, contestable, and enforceable in real time. ←filecite→turn17file3↑

Failure mode: governance only enforced off-screen.

---

## 10. Community Signals Informing DP5

Community submissions and aligned work point to several recurring signals:

- desire for publisher-controlled trust anchors, such as decentralized URIs under existing domains

- need for namespace rights and management for non-human entities and AI systems
- interest in hierarchical, semantically meaningful names independent of physical nodes
- support for trust-schema-driven semantics and context-aware permissions
- demand for name chains as resolvable trust anchors for people, objects, and assertions
- need for decentralized identifiers and tags for posts, paths, and navigation objects
- concern that community identifiers may be seized, censored, or captured by platforms

These signals indicate that DP5 is not only about naming people. It is about naming the structure of the Metaweb itself.

---

## 11. Non-Goals and Explicit Boundaries

DP5 does not:

- require one global namespace for all contexts
- replace DNS, DIDs, ENS, SNS, BRC333, ordinals, or publisher-controlled URIs
- guarantee that all valuable names will be available
- eliminate disputes, squatting, or fraud completely
- treat tradeability as superior to stewardship
- require real-name identity
- collapse human, AI, organizational, and artifact namespaces into one undifferentiated model

DP5 defines conditions under which naming remains interoperable, governable, and accountable across systems.

---

## 12. Minimum DP5 Alignment (Upgraded Baseline)

This section is upgraded from descriptive to **testable compliance criteria**.

A system claiming DP5 alignment **MUST** pass the following checks:

---

### 12.1 Deterministic naming

- Canonical forms are defined and machine-verifiable
- Validation produces identical results across implementations

Test: same input → same validity result everywhere

---

### 12.2 Multi-source resolution

- Names resolve via more than one possible source
- Resolver provenance is exposed



Test: user can inspect where resolution came from

---

### 12.3 Explicit state signaling

Every name MUST expose state:

- verified
- provisional
- disputed
- quarantined
- forked

Test: UI or API returns state metadata

---

### 12.4 Ownership auditability

- Current controller is identifiable
- Transfer history is preserved

Test: ownership lineage query returns full chain

---

### 12.5 Portability

- Names function across at least 2 independent systems

Test: same name resolves meaningfully in multiple contexts

---

### 12.6 Anti-spoofing safeguards

- System detects or flags similarity-based impersonation

Test: registering visually similar name triggers warning or constraint

---

### 12.7 Registry memory

- Historical snapshots exist with integrity proofs

Test: past state can be reconstructed with checksum validation

---

### 12.8 Dispute mechanism

- Users can initiate and observe disputes

Test: dispute creates visible state transition

---

## 12.9 Non-human classification

- Agents and automated systems are distinguishable from humans

Test: entity type is explicitly encoded and exposed

---

These criteria transform DP5 from a principle into a **verifiable standard**.

---

These conditions define the minimum viable namespace layer of the Meta-Layer.

---

## 13. Open Questions and Future Work (Refined)

DP5 now identifies key frontier problems:

### 13.1 Cross-namespace interoperability

How do independent naming systems interoperate without collapsing into monopoly or fragmentation?

---

### 13.2 Semantic portability

How can meaning travel with names across cultural, linguistic, and technical contexts?

---

### 13.3 Anti-squatting mechanisms

What mechanisms balance open access with protection against speculative enclosure?

---

### 13.4 AI-native identity

How should agent identities evolve as they gain autonomy, persistence, and economic activity?

---

### 13.5 Namespace governance forks

What legitimacy models determine when a community can fork naming rules?

---

## 13.6 Human-readable vs machine-secure naming

How do we balance usability with cryptographic robustness?

---

## 13.7 Unicode and global inclusion

How do we support multilingual naming without increasing spoofing risk?

---

## 13.8 Economic design

What pricing, staking, or decay mechanisms prevent hoarding while preserving ownership?

---

These questions define the path toward ML-RFC maturation.

---

## 14. Relationship to Other Desirable Properties

DP5 is foundational and interdependent.

- **DP1** depends on identifiers that bind identity and accountability without forcing platform control
- **DP2** depends on participant control over names, handles, personas, and namespace portability
- **DP4** depends on identifiers that preserve data provenance without forcing cross-context correlation
- **DP6** depends on tradeable meta-assets, domains, and artifacts that preserve ownership and settlement integrity
- **DP7** depends on canonical forms and resolution semantics that work across systems
- **DP9** depends on attribution and contribution identifiers that cannot be trivially spoofed or replayed
- **DP12–DP13** depend on agent and tool identifiers that can be constrained, audited, and governed
- **DP14–DP15** depend on provenance and transparency for names, artifacts, and registry changes
- **DP18–DP20** depend on community and ownership identifiers that persist across governance and migration

A failure in DP5 propagates upward. If names cannot be trusted, ownership cannot be trusted, identity fragments, and interoperability becomes deception.

---

## 15. Path Toward ML-RFC

Progression toward ML-RFC maturity should include:

- standardized canonical forms for meta-domains, personal identifiers, and artifact identifiers
- shared validation rules and reserved-name policies
- resolver and registry schemas
- dispute-state semantics
- transfer receipt formats

- registry snapshot standards with checksums
- conformance tests for resolution integrity, ownership binding, and namespace portability
- implementation evidence from registry prototypes and overlay applications

Stable portions may be promoted first, especially canonical form rules, validation logic, and registry state semantics.

---

## 16. Closing Orientation

DP5 is the claim that participants, communities, agents, and artifacts deserve names they can carry across the Meta-Layer.

Without sovereign names, identity is rented, ownership is fragile, and discovery remains platform-shaped.

With DP5, people can claim space, communities can preserve continuity, artifacts can be addressed, and the Metaweb can become navigable without surrendering naming power to a single platform.

DP5 turns names into civic infrastructure.

To claim your space in the Meta-Layer is not merely to register a label. It is to anchor presence, meaning, and accountability in a shared world.

---